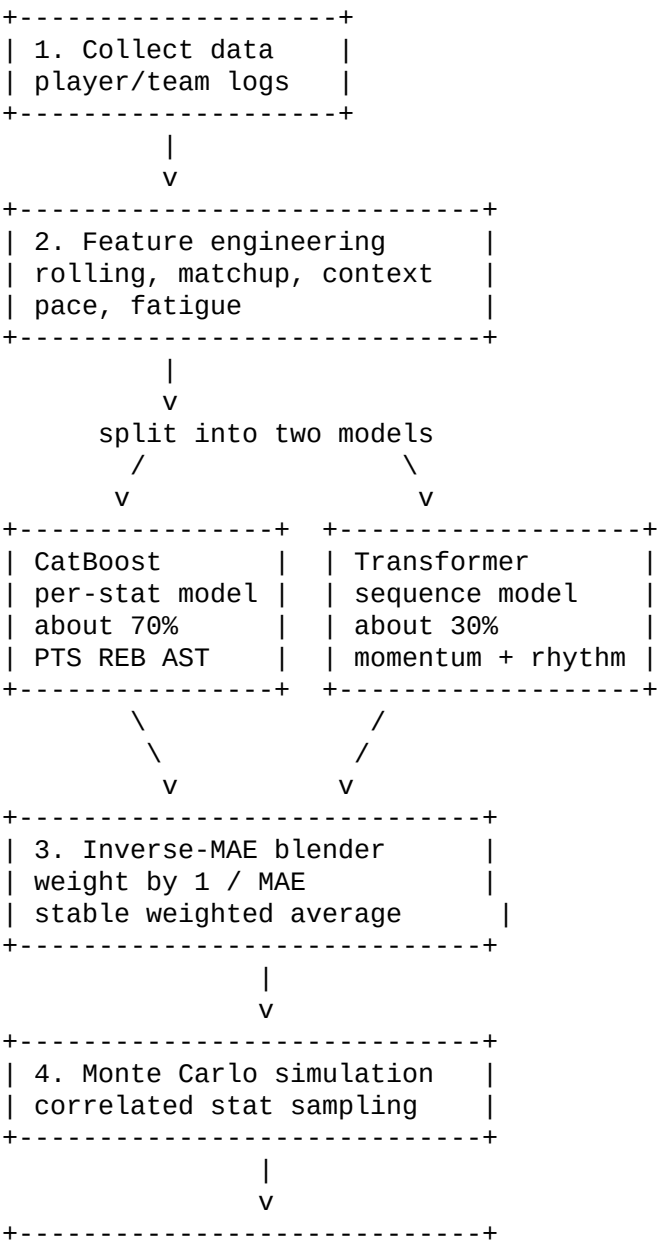


Recommended Three-Model Stack

Simple step-by-step guide that matches the diagram

Generated on 2026-03-26

This PDF explains the picture in plain language. It follows the same flow as the SVG: collect data, build features, split into CatBoost and Transformer, blend them by inverse MAE, run Monte Carlo simulation, and produce player projections.



5. Player projections	
mean, mode, CI, over/under	
+-----+	

What each step means

Collect data means downloading player logs and team logs.

Feature engineering means turning raw stats into model inputs like rolling averages and matchup data.

CatBoost gives one prediction for each stat such as points, rebounds, assists, steals, blocks, and turnovers.

The Transformer looks at game order, so it can learn recent rhythm and momentum.

The blender combines both model outputs and gives more weight to the one with lower MAE.

Monte Carlo simulation repeats the game many times so the final output is not just one number.

Step 1: Collect data

The first box in the diagram is data collection. The program reads NBA player logs and team logs, then saves them as CSV files under `data/`. This is the raw material for everything else.

The updater gets fresh game logs from NBA.com Stats API.

It keeps player rows and team rows in simple files so training can reuse them later.

If you run it again, it can only fetch new games instead of starting over.

Step 2: Feature engineering

The second box turns raw rows into useful signals. The code makes rolling averages, matchup stats, pace numbers, and context features like home or away, rest days, and fatigue.

This step makes the data easier for the models to read.

It uses only past games, so the model does not peek into the future.

This is where the program gets most of its predictive power.

Step 3: Two model paths

After features are ready, the flow splits into two paths. CatBoost handles the main per-stat prediction. The Transformer handles sequence patterns across recent games. The diagram shows CatBoost at about 70 percent and Transformer at about 30 percent before blending.

CatBoost is strong for tabular data and one model is trained for each stat.

The Transformer is strong for order and context across the last several games.

The diagram also shows CatBoost with quantile variants, which help estimate uncertainty.

Step 4: Inverse-MAE blender

The two model outputs meet in the blender. Inverse MAE means the model with lower error gets more weight. So if CatBoost has been more accurate, it counts more. If the Transformer is better for a player, it gets more weight instead.

This keeps the final answer stable instead of trusting only one model.

MAE stands for mean absolute error, which is a simple measure of past mistakes.

The blender is the merge point in the diagram.

Step 5: Monte Carlo simulation

After blending, the program does many simulations. This is how it turns one estimate into a range of outcomes. The simulator keeps the stat groups linked together so the results look realistic.

One game is simulated many times, not just once.

Points, rebounds, assists, steals, blocks, and turnovers move together in a realistic way.

This makes the final projection more useful than a single raw prediction.

Step 6: Player projections

The last box is the output. The program writes player projections with a mean, a mode, a confidence interval, and over/under style results. That output is what the query tool and season simulator use later.

`query_prob.py` reads those saved projections and answers simple betting questions.

`simulate_season.py` uses the same model stack to make game and season reports.

The diagram is a simplified picture of that full flow.

Very short summary

Data goes in.

Features are built.

CatBoost and the Transformer make predictions.

The blender combines them.

Monte Carlo simulation makes the result realistic.

Player projections come out the other side.

Files to look at

update_data.py for data collection.

src/training/pipeline.py for the CatBoost + Transformer training flow.

src/models/model_manager.py for blending and prediction.

src/simulation/game_simulator.py for Monte Carlo simulation.

query_prob.py for the final question-and-answer tool.